Executable JavaScript as a Specification Language: A JS-SAM Case Study on SysMoBench

Jean-Jacques Dubray
jdubray@gmail.com

In collaboration with the SysMoBench / Specula team

July 2, 2026 (draft)

Abstract

SysMoBench grades LLM-generated specifications of real concurrent and distributed systems through four automated phases: syntax, bounded model checking, conformance against traces captured from the running system, and invariant verification. We extend it with JS-SAM, its first non-formal-language backend — JavaScript specifications in the SAM pattern, whose semantics deliberately mirror TLA$^+$ — to test whether LLMs model systems more faithfully in a language abundant in their training data. We instrument the Asterinas kernel spinlock for ground-truth traces and evaluate four Claude models end to end, culminating in a pre-registered, paired comparison completed after review to a full 3×2 factorial: contract (SAM / lean / TLA$^+$) × prompt (with / without spec-level semantics), N=5 generations per cell.

Trace conformance is the only phase that discriminates (0–86.4% on state-changing windows, against an identity base rate the weakest model exactly matches; the other phases are unanimous, one pass provably vacuous), and one round of trace feedback repairs some specifications completely — audited as root-cause fixes, not memorization. A default-prompt evaluability gap (JavaScript 4/4 vs. TLA$^+$ 1/4) decomposes into a known TLC idiom trap and a harness fault, and vanishes under a one-line guardrail. The factorial then resolves the language question in an unexpected place: the specification *contract* is the dominant, uniform factor. A bare `next(state, action, model)` function, structurally isomorphic to the TLA$^+$ relation, ties TLA$^+$ at 100% for every model *with or without spec-level semantics in the prompt*, while prompt prescriptiveness — large enough to masquerade as a language effect — moves only two of four models within the SAM contract. The single cross-language split (derivation mode) traces to the conformance oracle's totality requirement penalizing idiomatic enabling-condition TLA$^+$; scored paradigm-fairly, it too is a tie. The result is validated by 40/40 model-generated lean-contract specifications passing every phase across two tasks. On the tasks tested, contract shape came first, the prompt second (a real but model-dependent factor), and the language nowhere — a conclusion conditional on the spinlock factorial pending richer-task replication. We report a single-system case study with its corrections documented in-line.

1 Introduction

Formal specifications are among the most effective tools for establishing the correctness of concurrent and distributed systems [7], and among the least used: writing them demands rare expertise and sustained effort. LLMs raise an obvious question — can they write specifications

for us? — and a harder one: *how would we know if they got it right?* A generated specification can be syntactically flawless, explore cleanly under a model checker, satisfy plausible invariants, and still describe a system that does not exist. The SysMoBench benchmark [1] confronts this directly. Its authors observed that when asked to model Etcd's Raft implementation, a frontier LLM produced, in effect, the specification from the appendix of the Raft paper — a "textbook" model with little connection to the implementation at hand [2]. SysMoBench therefore grades specifications in four phases of increasing demand: (1) syntax, (2) runtime model checking, (3) *transition validation* — replaying $(pre, action, post)$ windows captured from real executions against the generated model — and (4) invariant verification.

SysMoBench was built for TLA$^+$ [3], and has since grown a pluggable backend interface with Alloy [5] and PAT/CSP# [6] backends. All three are formal languages, and all three are rare in LLM training corpora relative to mainstream programming languages. This asymmetry motivates the extension we present here.

The JS-SAM hypothesis. SAM [9, 10] is a software-engineering pattern whose semantics are explicitly derived from TLA$^+$: *actions* compute proposals from inputs, a *model* accepts or rejects proposals as the sole locus of mutation, and *state* is a pure function of the model. A SAM specification is an ordinary executable JavaScript module. JavaScript is plausibly the single most abundant programming language in LLM training data. The hypothesis, then:

> *Can an LLM produce a correct specification of a real system more readily in a pattern hosted by a language it has seen constantly in training (JavaScript), than in formal languages it has rarely seen — while preserving the TLA$^+$-style discipline that makes the artifact checkable?*

Either answer is informative. If JS-SAM scores diverge upward, host-language familiarity dominates and formal syntax is a bottleneck; if scores match or fall, the hard part is modeling, not notation.

Contributions. This paper reports the first end-to-end results from the JS-SAM backend, all on the SysMoBench `spin` task (the Asterinas OS spinlock [12]):

1. **A non-formal-language backend for SysMoBench** (§3): the JS-SAM module contract, a sandboxed execution helper, and — a first for the benchmark — a *direct* Phase-3 replay path requiring no coding-agent orchestration, because SAM's step relation *is* $model.present(action(data))$.

2. **A reproducible kernel trace-capture methodology** (§4): instrumenting the real Asterinas spinlock, driving a two-thread contention scenario in a single-CPU kernel test without deadlock, and folding low-level kernel events into model-independent $(pre, action, post)$ windows — producing the 28-window corpus used throughout.

3. **A four-model comparison** (§5) showing that Phases 1, 2, and 4 have no discriminating power on this task while Phase 3 spreads the models from 0% to 86.4% on state-changing windows (21.4% to 89.3% headline, against an identity base rate of 21.4%) — and that general model capability does not predict modeling accuracy: the most capable model tested produced a specification that no-ops every lock release.

4. **A repair-loop experiment** (§6) demonstrating that self-correction from precise trace feedback is a distinct capability axis from one-shot accuracy — capable models repair fully (verified by a held-out generalization audit as fixes, not memorization), the weakest not at all, with mid-tier outcomes unstable at N=1.

5. **A JS-SAM-vs-TLA$^+$ evaluability comparison** (§7): 4/4 JS-SAM specifications reach end-to-end evaluability versus 1/4 for TLA$^+$ — reported strictly per cause (a known TLC idiom trap twice, a harness fault once) and shown to vanish under a one-line prompt guardrail: a default-prompt robustness observation, not a language-capability finding.

6. **A controlled cross-language conformance study** (§8): a pre-registered, paired design, completed after review to a full 3×2 factorial in (contract, prompt) (N=5 generations per cell), pinning both languages to the same observable-state contract and replaying the same trace corpus mechanically in both. The completed factorial attributes the conformance gap to the SAM contract's machinery — not to the JavaScript language, not to formality, and not (as an incomplete design had suggested) primarily to prompt prescriptiveness: the lean contract reaches 100% for every model *without* the semantics block, while the semantics block within SAM moves only two of four models. This yields a concrete design change for the backend, validated by 40/40 lean specifications passing every phase across two tasks (§8.4).

In light of the results, we regard the trace-capture methodology (2), the controlled-comparison methodology (6), and the contract-shape finding as the primary contributions; the JS-SAM backend (1) is the vehicle that surfaced them, and its own results argue for the lean revision of its contract.

We emphasize scope honestly: the controlled experiments concern one system — the spinlock — and a 28-window trace corpus (the lean-contract validation extends to a second task with its own 60-window corpus); Experiments 1–3 are single-generation, with N=5 replication in Experiment 4 and the lean studies. We present it as a case study whose value lies in the precision of the failure attribution and the reproducibility of the pipeline, not as a settled verdict on the hypothesis (§10). Corrections made under review are kept in-line where they changed a claim — the corrections are themselves findings — at a deliberate cost in linearity; readers wanting only the current claims can rely on the tables, §9, and §12.

2 Background

2.1 SysMoBench

SysMoBench [1] pairs eleven real systems (concurrent synchronization primitives and distributed protocols, from the Asterinas spinlock to Etcd Raft) with instrumentation harnesses and expert-written invariants. Given the system's source, an LLM generates a specification, which is scored in four automated phases:

Phase 1 — Syntax. Does the specification parse and structurally validate? (For TLA$^+$: the SANY analyzer.)

Phase 2 — Runtime model checking. Does bounded state-space exploration terminate without errors, deadlocks, or nondeterminism? (For TLA$^+$: TLC [4].)

Phase 3 — Transition validation (conformance). Execution traces from the *real* system are cut into windows $(pre, action, post)$; each window passes if the generated model, started at *pre* and applying *action*, produces *post*. The output is a per-action scorecard.

Phase 4 — Invariant verification. Expert invariants (e.g., mutual exclusion) are translated into the specification's language and checked.

The benchmark's central published finding is that frontier LLMs cluster near 100% on syntax but average roughly 46% on conformance and 41% on invariants [2]: models write valid TLA$^+$ fluently yet describe the textbook protocol rather than the implementation. Phase 3 exists precisely to distinguish "writing TLA$^+$" from "modeling this system."

2.2 The SAM pattern

SAM (State–Action–Model) [9, 10] factors a program's temporal behavior the way TLA$^+$ factors a specification. An *action* is a pure function from inputs to a *proposal*; the *model* alone decides whether to accept a proposal and mutate itself (the analogue of a TLA$^+$ next-state relation, with acceptors playing the role of guards); *state* is a pure function of the model. A step of the system is $model.present(action(data))$. The pattern is implemented by the `sam-pattern` library [11], which also provides a bounded explorer (`checker`) over the model's reachable states; in the JS-SAM backend, this checker is what executes Phases 2 and 4.

Two properties make SAM suitable as a specification substrate. First, the semantic alignment with TLA$^+$ means the benchmark's phases map onto it without distortion: bounded exploration is Phase 2, replaying a trace window is literally one SAM step, and invariants are predicates over $getState()$. Second, the artifact is an executable JavaScript module: there is no separate tool chain between the specification and its own evaluation.

3 The JS-SAM backend

SysMoBench backends implement a `LanguageBackend` interface and register themselves; the CLI and the four evaluators resolve them by name with no further plumbing. The JS-SAM backend consists of a Python adapter and a small Node.js helper (`tools/js-sam/cli.mjs`) with four subcommands — `validate`, `check`, `transitions`, `invariants` — one per phase.

Module contract. The generation prompt instructs the model to emit a single CommonJS module:

```
module.exports = {
  instance,       // SAM instance (synchronous steps)
  init,           // () => void : reset to the initial state
  actions,        // { AcquireLock: (data)=>void, ReleaseLock: (data)=>void }
  getState,       // () => plain JSON-serializable snapshot
  setState,       // (snapshot) => void : force a state (Phase 3 replay)
  checkerIntents, // action descriptors + input domains (Phases 2 & 4)
};
```

`actions` must cover exactly the task's target actions; `getState` must return a plain object (this is what trace states are diffed against); `setState` exists solely so Phase 3 can replay a window

without searching for a path to its pre-state; and the module must be deterministic and I/O-free. `checkerIntents` declares each action's finite input domain — for `spin`, every $\{thread, callType\}$ combination — playing the role of a TLA$^+$ constants block. Notably, there is *no separate configuration artifact*: the module is self-contained, a fact that becomes material in §7.

Sandboxing. Model-generated JavaScript is untrusted. Every helper invocation runs in a throwaway `node:20-slim` Docker container with no network, a read-only root filesystem, a non-root user, and CPU/memory/PID caps. The container is the trust boundary.

The direct Phase-3 path. For TLA$^+$, SysMoBench's transition validation drives TLC through a coding-agent workflow that constructs a checking module per window. JS-SAM needs none of that: the evaluator loads the module, and for each window executes `setState(pre)`; `actions[name](data)`; and diffs `getState()` against *post* under the benchmark's projection rule (only keys present in the trace's post-state must match, so model-internal bookkeeping is ignored). This "direct path" is the first non-agent Phase-3 implementation in the benchmark, and it follows from SAM's semantics: a window *is* a SAM step.

Cross-platform engineering. Making the pipeline run on a Windows host (in addition to Linux/macOS) required fixing several host assumptions in the harness — POSIX-only `os.getuid()` calls, host-path container mounts invalid for `C:\` paths, and console encoding — plus threading the helper's `stepsExplored` metric through a field previously hardcoded to zero for non-TLA$^+$ backends. We also generalized the benchmark's agent-based invariant translator (previously TLA$^+$-only) into a shared, language-neutral component that JS-SAM can use, although all experiments below use the direct translator for parity across languages. These changes are upstreamed as pull requests.

4 Capturing ground truth: kernel traces for Phase 3

Phase 3 is the only phase requiring data external to the model, and SysMoBench does not ship trace corpora: traces are generated on demand by instrumenting the real system. Additionally, the maintainers' own `spin` captures use three threads while the JS-SAM prompt models two, so the backend required its own corpus at the granularity it models. This section documents the pipeline; it is the experimental setup's main methodological contribution and is fully reproducible from a single script.

4.1 Pipeline

1. **Pin a compatible revision.** The reference instrumentation patch applies cleanly at Asterinas v0.16.0 (matching the toolchain in the `asterinas/asterinas:0.16.0` image), not at `main`.

2. **Instrument.** Apply the benchmark's reference patch, which adds a trace module to the kernel's spinlock emitting one serial JSON event per lock operation, plus a new two-thread kernel test (below).

3. **Build and run.** Inside the container: build the initramfs and run the instrumented kernel test under QEMU (software emulation), collecting the event stream from the serial port.

4. **Parse.** Fold the event stream into (*pre*, *action*, *post*) NDJSON windows.

4.2 A two-thread contention scenario without deadlock

The reference spinlock kernel tests are single-actor: every operation is attributed to one thread, so contention — the behavior that makes a lock interesting — never appears in the trace. But a blocking `lock()` from a second actor in a single-CPU kernel test would spin forever. The scenario therefore expresses the second actor's contention through non-blocking `try_lock` calls that fail without blocking:

```
actor0.lock()      -> TryAcquireBlocking(0), AcquireSuccess(0)  // 0 holds
actor1.try_lock()  -> AcquireFail(1)          // contention, no deadlock
actor0.release()   -> Release(0)
actor1.lock()      -> TryAcquireBlocking(1), AcquireSuccess(1)  // 1 holds
actor0.try_lock()  -> AcquireFail(0)          // contention
actor1.release()   -> Release(1)
actor0.try_lock()  -> AcquireSuccess(0)       // try from free succeeds
actor0.release()   -> Release(0)
```

A second, longer scenario — more alternation of the holder, more failed and successful `try_lock` attempts, and same-thread reacquisition — extends the corpus from 8 windows to **28 windows** (17 `AcquireLock`, 11 `ReleaseLock`), the corpus used in all experiments below.

4.3 Model-independent state windows

The kernel emits low-level events (`TryAcquireBlocking`, `AcquireSuccess`, `AcquireFail`, `Release`). The parser folds these into windows keyed on the *objective* spinlock state — $\{lockHeld, lockHolder\}$, the real observable ownership — and relies on the projection rule to ignore any model's internal bookkeeping (thread status, call type). A failed acquire under contention produces a window whose post-state ownership is unchanged; a successful acquire records the new holder together with whether it arrived via the blocking (`lock`) or non-blocking (`try`) path. Keying the corpus on observable state keeps the ground-truth oracle independent of any model under test.

5 Experiment 1: four models, four phases

5.1 Setup

Four Claude models spanning the capability range — Haiku 4.5, Sonnet 4.6, Opus 4.8, and Fable 5 (Anthropic's most capable model at the time of writing) — each generated one JS-SAM specification for `spin` via a single prompt (`direct_call`), and the same artifact was scored in all four phases. Phase 2 and 4 exploration ran through the `sam-pattern` library's bounded checker at the task's default depth bound of 6; Phase 3 used the 28-window corpus; Phase 4 checked three expert invariants (MutualExclusion, LockStatusConsistency, NoDeadlock) translated via the direct API path.

5.2 Results

Two results stand out.

Table 1: Four-phase JS-SAM results on `spin` (28-window corpus). P3 per-action columns give windows passed / windows of that action. Base rate: 6 of the 28 windows (all contention acquires) have post = pre, so the identity function scores 21.4% overall and 6/17 on acquires — Haiku's row equals that base rate exactly; conditional on the 22 state-changing windows the P3 column reads 86.4 / 36.4 / 36.4 / 0%.

Model	P1	P2 (distinct states)	P3 overall	P3 Acquire	P3 Release	P4
Claude Opus 4.8	pass	pass (7)	**89.3%** (25/28)	14/17	11/11	3/3
Claude Fable 5	pass	pass (7)	**50.0%** (14/28)	14/17	**0/11**	3/3
Claude Sonnet 4.6	pass	pass (7)	**50.0%** (14/28)	14/17	0/11	3/3
Claude Haiku 4.5	pass	pass (**1** — vacuous)	**21.4%** (6/28)	6/17	0/11	3/3

Only the reality check discriminates. Every model passes Phase 1, Phase 2, and Phase 4. On this task those three phases have *zero* discriminating power. An earlier draft reported the identical 326,592 "states explored" per model as confirming structurally equivalent state spaces; an audit prompted by review showed that number is the checker's *step count* — safety-callback invocations over the intent-permutation tree, $(d{+}1) \cdot 6^d = 7 \cdot 6^6$ — a pure function of the contract-pinned intent domain and depth bound, identical for any conforming spec and therefore model-independent. Counting *distinct semantic states* (unique model snapshots) instead separates the specs: 7 for Opus, Fable, and Sonnet, but **1 for Haiku**, whose intent actions drop their arguments so every proposal is rejected — its exploration never leaves the initial state. Haiku's Phase-2 pass is thus *vacuous* (no crash, deterministic, and serializable hold trivially for a spec that cannot move), and its Phase-4 invariants are satisfied over that single state. Phase 2 verifies checkability, not semantic adequacy; the distinct-state count, now reported by the checker, is the semantic signal. Phase 3 spreads the models across a 4× range, and — restricted to comparisons it can make — orders Opus above the mid tier above Haiku. A base-rate audit prompted by review sharpens the spread: 6 of the 28 windows (all contention acquires) have post = pre, so the *identity function* scores 21.4% overall and 35.3% on acquires. Haiku's row is exactly that base rate — its "Acquire 6/17" is precisely the six freebies, and it passes *zero* windows requiring a state change — so conditional on the 22 change windows the spread is not 21.4–89.3% but **0–86.4%** (Opus 19/22, Fable and Sonnet 8/22, Haiku 0/22). Every arm of every model collects all six freebies; the conditional-on-change rate is the discriminating metric and accompanies headline rates hereafter. This independently reproduces, in a non-formal language, the central SysMoBench finding for TLA$^+$: internal consistency is cheap; conformance with the real system is where specifications fail [2].

General capability does not predict modeling accuracy. The most striking row is Fable 5, the most capable model tested, scoring 50% — below Opus 4.8 and tied with mid-tier Sonnet. The cause is specific and systematic, not noise: Fable's specification handles acquisition competently (14/17) but *no-ops every single release* (0/11). After any `ReleaseLock`, its model leaves the lock held:

```
ReleaseLock  expected {lockHeld: false, lockHolder: null}
             got      {lockHeld: true,  lockHolder: 0}
```

The release logic *reads* correctly in isolation — it guards on the holder, then clears the state — but on replay the release never takes effect: a categorical, whole-action modeling defect. Crucially, this specification passed syntax, completed the full bounded exploration (7 distinct

states) without violation, and satisfied all three invariants, including mutual exclusion. (Its release *does* fire when reached from the initial state, where its auxiliary `threadStatus` guard holds; the defect is that replay pins only the observable pre-state, leaving the guard unsatisfiable — so exploration from init cannot expose it.) Only replay against the real kernel's transitions caught it.

The recurring failure among the passing acquisitions is also precise: all three 14/17 models fail exactly the three `try_lock`-from-free windows — the non-blocking acquire that should succeed on a free lock. Blocking acquisition (abundant in training data and in textbook treatments) is modeled correctly by every model except Haiku; the less common non-blocking path is where they slip. This echoes, at micro scale, SysMoBench's "textbook modeling" diagnosis: models reproduce the familiar template and miss the implementation-specific path.

6 Experiment 2: repair from trace feedback

6.1 Setup

If a model is shown *exactly* which transitions its specification got wrong — the pre-state, the action and its data, the real system's post-state, and what the model produced instead — can it fix the specification? For each model we ran one repair round: score the original specification on Phase 3; build a repair prompt containing the full original module plus every failing window; ask the model to rewrite the module; confirm the result still passes Phase 1; re-score Phase 3 on the same 28-window corpus.

6.2 Results

Table 2: One-round repair from Phase-3 feedback (same 28-window corpus; original run — the generalization-audit replication differs for Sonnet, which repaired to 28/28 there).

Model	Baseline P3	Repaired P3	Acquire (base→rep)	Release (base→rep)
Claude Opus 4.8	89.3% (25/28)	**100%** (28/28)	82% → 100%	100% → 100%
Claude Fable 5	50.0% (14/28)	**100%** (28/28)	82% → 100%	**0% → 100%**
Claude Sonnet 4.6	50.0% (14/28)	60.7% (17/28)	82% → 100%	0% → 0%
Claude Haiku 4.5	21.4% (6/28)	21.4% (6/28)	35% → 35%	0% → 0%

Self-correction separates the models — but the middle tier is noise. In the run of Table 2, Opus and Fable repair to a perfect 28/28 in a single round, Sonnet repairs *partially* (it fixes the `try_lock` acquisition defect but not its release path), and Haiku does not improve at all — superficially a clean three-tier gradient (full / partial / none). A replication run (part of the generalization audit below, with artifacts saved) revised this: *Sonnet also repaired to 28/28.* The boundary that is stable across both runs is two-tier — Opus, Fable, and (unstably) Sonnet can exploit transition-level feedback; Haiku cannot — and single-round repair outcomes for mid-tier models are sampling-dependent at N=1.

The Fable reversal. Fable's baseline looked no better than Sonnet's, and its failure mode — a whole action silently broken — looked worse. Yet under repair, Fable fully recovers: its

release defect was a *recoverable slip*, not a capability ceiling. One-shot specification accuracy and repair-from-feedback are different axes of capability, and a benchmark that measures only the former will misrank models for any workflow that includes a correction loop — which realistic agentic formal-modeling workflows do [2]. (An earlier draft contrasted Fable's recovery with Sonnet's failure to fix the identical symptom; the replication run removed that contrast.)

Generalization audit: the repairs are fixes, not memorization. Repaired specifications were originally re-checked only on Phases 1 and 3, on the same 28 windows quoted in the repair prompt — leaving open that a repair might *memorize* the failing windows (branch on the pre-state, return the expected post), which would make this an experiment about prompt-following rather than modeling. An audit re-ran the repair loop with artifacts saved and scored each baseline and repaired specification on the seen corpus, on *held-out* windows, and on Phases 2 and 4. The 28 seen windows collapse to 8 distinct (pre-state, action, data) combinations; the held-out set is the 10 combinations of the observable domain the corpus never shows (acquire on a held lock, release by a non-holder, release of a free lock). All repaired specifications pass all held-out windows, reach the same 7 distinct semantic states as a correct specification under bounded exploration (Haiku's unrepaired specification remains at 1), and hold all invariants. Direct inspection found no memorization — no state-equality tables or window literals; the successful repairs are the same root-cause fix, moving the release ownership check from the auxiliary `threadStatus` guard (unsatisfiable under a pinned observable pre-state) to the authoritative `lockHeld`/`lockHolder`, with Sonnet's repair documenting the diagnosis in a comment. One structural honesty note: because the kernel instrumentation emits acquire events at acquisition success, every held-out combination is an observable no-op, so held-out scoring alone refutes only memorizers with unsafe defaults — the inspection and Phase-2/4 evidence carry the conclusion, and a state-changing held-out set (as `locksvc` affords) is the right vehicle for a stronger version of this audit.

What this does and does not measure. The repair prompt contains the failing windows themselves, so the experiment measures "can the model exploit precise, correct feedback," not blind improvement. That is the intended question — it is exactly the signal available inside an agentic repair loop — but it is not a from-scratch re-test. A multi-round follow-up (run for review point 9; `scripts/repair_multiround.py`) answers the convergence question: Haiku does *not* converge — three rounds of full-failure feedback leave it at 6/28 with its exploration still confined to one state (6→6→6; every round's spec saved and audited) — while Sonnet full-repairs in round one again (its second full repair in three runs, further evidence the original "partial" tier was a sampling artifact).

7 Experiment 3: JS-SAM versus TLA$^+$

7.1 Setup

The head-to-head the backend exists to inform: the same four models, the same `spin` task, one `direct_call` generation per model per language, scored on Phase 1 (syntax: SANY vs. module validation), Phase 2 (model checking: TLC vs. bounded exploration), and Phase 4 (invariant verification, direct translator for both languages). Phase 3 is deliberately *not* compared: JS-SAM's Phase 3 is a self-contained direct replay while TLA$^+$'s is a heavyweight coding-agent path — different mechanisms, not a fair one-to-one. Phase 4 is compared as pass/fail because

the two languages ship different invariant-template counts for `spin` (seven for TLA$^+$, three for JS-SAM) — and the JS-SAM three are safety-only, a qualitative weakness examined in §10: as implemented, JS-SAM Phase 4 could not have failed the never-releasing defect of §5 in principle.

7.2 Results

Table 3: JS-SAM vs. TLA$^+$ on `spin`: models producing a passing artifact per phase.

	P1 syntax	P2 model-checkable	P4 invariants
JS-SAM	4/4	**4/4**	**4/4**
TLA$^+$	4/4	**1/4**	**1/4**

Table 4: Per-model TLA$^+$ detail.

Model	P1	P2	P4	P2 failure cause
Claude Opus 4.8	pass	**fail**	fail	unbounded `CHOOSE` (TLC rejects)
Claude Fable 5	pass	pass (159 distinct states)	pass (7 inv.)	—
Claude Sonnet 4.6	pass	**fail**	fail	unbounded `CHOOSE` (TLC rejects)
Claude Haiku 4.5	pass	**fail**	fail	config generation fell back; no usable `.cfg`

Syntax is not where the languages differ. Every model writes syntactically valid specifications in *both* languages — consistent with SysMoBench's finding that frontier models have essentially saturated TLA$^+$ syntax [2]. The divergence is at Phase 2: whether the specification is actually *checkable*. All four JS-SAM modules run; one of four TLA$^+$ specifications does. Each TLA$^+$ failure is attributable to a specific structural cause.

Cause 1: a TLA$^+$ semantic pitfall JavaScript does not have. Opus and Sonnet both modeled "no lock owner" as `NULL == CHOOSE v : v \notin Threads`. SANY accepts this — syntax passes — but TLC cannot enumerate a `CHOOSE` whose bound variable ranges over an unbounded domain (`CHOOSE` itself is deterministic), so model checking fails. The idiom is natural, looks correct, and is a well-known TLC trap — though the indictment is of LLM TLA$^+$ fluency rather than of the language: no experienced TLA$^+$ author models absence this way, and agentic workflows already write around it in practice [13]. JavaScript has a native `null`, so the corresponding JS-SAM specifications express the same concept with no trap available to fall into. This structural hazard caught two of four models, *including the model with the best JS-SAM conformance score* (Opus). It is a clean instance of the hypothesis's mechanism: the failure is not "the model cannot model a spinlock" (its JavaScript model of the same system scored 89.3% against real traces) but "the formal language exposes a semantic surface on which competent models slip."

Cause 2: a configuration surface JS-SAM does not have. TLA$^+$ model checking requires a separate `.cfg` artifact (constants, init/next, invariants) that the harness must generate to match the specification; for Haiku, that generation fell back and produced no usable configuration (zero states explored). A JS-SAM module is self-contained and executable — there is no second

artifact to generate, so this entire failure mode is structurally absent. We flag the attribution honestly: the `CHOOSE` failures belong to the models; the Haiku failure belongs to the harness. But the asymmetry itself is structural — one language's evaluability depends on a fragile secondary artifact and the other's does not.

Net. On this task, models reach an end-to-end evaluable specification far more readily in JS-SAM (4/4 through Phase 4) than in TLA$^+$ (1/4). Read narrowly, this is evidence for the hypothesis in its practical form: *host-language familiarity plus a self-contained executable artifact lowers the barrier to a usable specification.* It does not, by itself, show that JS-SAM specifications are more *faithful*; it shows they are far more often checkable at all, which is the precondition for faithfulness to be measured. And given the per-cause attribution above and the guardrail result of §8, we treat the 4/4-vs-1/4 figure throughout this paper as a within-experiment observation about default-prompt robustness, not a conclusion about the languages. Experiment 4 (§8) measures faithfulness directly, under a design built to make the two languages comparable — and resolves it in an unexpected place.

8 Experiment 4: cross-language conformance under a controlled contract

Experiment 3 could not compare Phase 3 across languages because the two backends implement it by incomparable mechanisms: JS-SAM runs a mechanical replay, while TLA$^+$ runs a coding-agent path — necessary because a free-form TLA$^+$ specification invents its own variable names, so mapping trace states onto spec states requires interpretation. A naive comparison would measure the replay machinery, not the specifications. This experiment removes that asymmetry with a pre-registered, paired design; the full methodology note, written to be socialized with the benchmark maintainers, is included in the artifact repository.

8.1 Design

Equalize the observable-state contract. The degree of freedom that forces TLA$^+$ onto the agent path — free choice of variables — is one the JS-SAM prompt already removes by pinning `getState()`'s shape. We remove it symmetrically: a constrained TLA$^+$ prompt mandates `VARIABLES lockHeld, lockHolder` (the trace schema), action operators `AcquireLock(thread, callType)` / `ReleaseLock(thread)`, and a TLC-safe sentinel (`NONE == "none"`, not `CHOOSE`). Both languages then answer the identical question — *express this system over this observable state* — and trace states map onto spec states with no interpretation.

A direct TLC replay path (no agent), functional on both sides. With the contract fixed, TLA$^+$ transition validation becomes mechanical, mirroring JS-SAM's `setState` → action → diff: for each window, a synthesized module pins the pre-state in its initial predicate and takes exactly one step of the window's action. Mere reachability of the traced post-state would be too weak a criterion: a TLA$^+$ action is a *relation*, and an over-permissive action can reach the correct post-state while also admitting wrong ones — passing an existential check that the functional JS replay (one `next` state, compared exactly) would fail. The replay therefore enforces the same functional relation on both languages: it enumerates the action's one-step image from the pinned pre-state and requires that the image be exactly the traced post-state (branching factor

1; over-permissiveness fails the window). The check was validated in three directions: a contract-following specification passes 28/28 (positive control); the same specification with `ReleaseLock` mutated to a no-op fails exactly the 11 release windows (negative control); and a deliberately permissive specification — holder set to *any* thread on acquire — passes 28/28 existentially but only 11/28 functionally, with branching factor 2 (permissive control), demonstrating that the functional check is strictly stronger and necessary. An audit of all 20 generated TLA$^+$ specifications found every one deterministic: existential and functional scores coincide on every window, and the maximum branching factor is 1. The TLA$^+$ results below are therefore earned under a relation as strict as the JS one, not inflated by existential slack.

Two commitments of this oracle must be named, because they are modeling-paradigm choices rather than neutral ground. First, requiring every trace event to map onto a named, *total*, successor-producing action is the lean/JS paradigm by construction. Idiomatic TLA$^+$ specifies *enabling conditions* and relies on stuttering ($[Next]_{vars}$ always admits $vars' = vars$): a failed acquisition is naturally a disabled action plus a stutter, which this oracle scores as *unscoreable* rather than as a legitimate no-op — penalizing enabling-condition specifications independent of their correctness. Wherever the cross-language comparison is drawn, we therefore report the conditional (paradigm-fair) score alongside the unconditional one, and we flag a stutter-aware oracle — a $post = pre$ window satisfiable by a stuttering step — as the fairer instrument for such specifications. Second, the functional branching-factor-1 requirement encodes a determinism assumption that holds here only because the trace scenarios are deterministic by construction; for genuinely concurrent systems a relational next-state is fidelity, not over-permissiveness, and the oracle must weaken to set-conformance (traced post-state in the one-step image, with tightness reported separately) before this methodology reaches Raft-class protocols.

Four arms, because the prompt and the contract are both confounds. An initial two-arm run (deployed JS-SAM prompt vs. constrained TLA$^+$) produced what looked like a decisive TLA$^+$ reversal. But the constrained TLA$^+$ prompt *states the exact single-step observable semantics* (free → acquire; held → unchanged; holder → release), which the deployed JS-SAM prompt does not: a difference in prompt content, not in language. And even with prompts equalized, the two substrates still differ in a second way — the SAM contract obliges an executable module with proposal/acceptor wiring and auxiliary state, while the TLA$^+$ specification is a bare two-variable relation. We therefore ran four arms initially: **JS(deployed)** — the shipped JS-SAM prompt; **JS(constrained)** — the deployed prompt plus the *identical* semantics block used by the TLA$^+$ arm; **plain-JS** — the same constrained semantics, but the specification is a bare `next(state, action, model)` transition function (argument names follow SAM usage) over only $\{lockHeld, lockHolder\}$: JavaScript, yet structurally isomorphic to the TLA$^+$ relation, with no SAM library and no auxiliary state; and **TLA(constrained)**. JS(constrained) vs. TLA(constrained) varies language and contract together; plain-JS vs. TLA(constrained) varies *only* the language; JS(constrained) vs. plain-JS varies *only* the contract.

Completing the factorial: the missing lean-without-semantics cell. Review observed that these arms form an *incomplete* 2×2 in (contract, prompt): the lean arm always carried the semantics block, so "contract minimality buys transcription fidelity" was established only conditional on spec-level semantics in the prompt, and any prompt-vs-contract effect ordering rested on the incomplete design. We added the missing cell — **plain-JS(no-semantics)**: the identical lean contract (module shape, two observable keys, action names, data schema, and a

description of the replay *mechanism*) with the entire single-step semantics block removed, so the model must derive the transition semantics from the Rust source alone. A mirrored TLA$^+$ derivation arm (the constrained TLA$^+$ contract, semantics block likewise removed) was added alongside it, completing the 3×2 of Table 5; its result — the study's only cross-language split — is analyzed in the findings below, with the full audit trail preserved in §10.

Replication, pairing, and checkability. N=5 generations per model per arm (80 in total), all scored per-window on the same 28-window kernel corpus. The pre-registered analysis was a pooled per-window McNemar test; review correctly identified that as *pseudo-replicated* — generations collapse to 1–2 unique behavioral fingerprints per (model, arm), so pooling 5×28 windows counts the same underlying defect up to five times and inflates any χ^2. The analysis of record is therefore at the generation level: we report per-arm uniqueness counts and an exact two-sided permutation test with the *generation* as the unit (statistic: difference in mean per-generation pass rate; all $\binom{10}{5} = 252$ relabelings; the smallest attainable p is $2/252 \approx 0.0079$, so starred results sit at the test's floor). Because the generations collapse to so few distinct behaviors, we present Δ as the primary quantity and read the permutation p-values as descriptive rather than as hypothesis tests: the effect sizes are large and uniform; the inferential machinery around them is at its resolution floor. The pre-registered checkability policy (conditional vs. unconditional scoring) proved moot in the constrained arms: *zero* windows were unscoreable in any constrained or plain-JS generation. (It becomes load-bearing in the derivation arms below.)

8.2 Results

Table 5: Cross-language Phase-3 conformance (mean unconditional pass rate over N=5 generations per cell; 28-window corpus). The columns form a full 3×2 factorial: contract (SAM / lean / TLA$^+$) × prompt (without / with the semantics block). All shortfall in the TLA$^+$-no-semantics column is *unscoreable* (guard-idiom partiality), not wrong post-states; conditional rates there are 100%.

Model	SAM, no sem.	SAM, sem.	lean, no sem.	lean, sem.	TLA$^+$, no sem.	TLA$^+$, sem.
Claude Opus 4.8	81.4%	89.3%	**100%**	**100%**	78.6%	**100%**
Claude Fable 5	57.9%	95.7%	**100%**	**100%**	**100%**	**100%**
Claude Sonnet 4.6	50.0%	**100%**	**100%**	**100%**	78.6%	**100%**
Claude Haiku 4.5	28.6%	40.7%	**100%**	**100%**	87.1%	**100%**

8.3 Findings

Prompt prescriptiveness is a large, real confound — but the completed factorial demotes it from the headline. Adding the same semantics block to the JavaScript prompt — changing nothing else — moved Fable from 57.9% to 95.7% and Sonnet from 50.0% to a perfect 100%. Had we reported the two-arm run, the headline would have been a decisive language reversal, and an earlier draft accordingly read "most of that effect was the prompt." The completed factorial shows that reading was an artifact of examining only the SAM column: at the generation level the within-SAM prompt effect is significant for exactly those two models (Fable Δ=.379, p=.024; Sonnet Δ=.500, p=.0079) and not for Opus (Δ=.079, p=1) or Haiku (Δ=.121, p=1), whereas the *contract* effect without any semantics block (SAM vs. lean, both promptless) is significant for *all four* models (Δ=.186/.421/.500/.714, each p=.0079). Prompt

Table 6: Generation-level analysis (replaces a pooled per-window McNemar table that review identified as pseudo-replicated; those χ^2 values are retracted). "Unique" = distinct behavioral fingerprints (per-window outcome vectors) among the 5 generations of each arm, ordered deployed-JS / constrained-JS / plain-JS / TLA$^+$. Δ = difference in mean per-generation pass rate vs. TLA$^+$; p from the exact permutation test (* = $p < 0.05$; 0.0079 is the smallest attainable value; given 1–2 unique behaviors per cell, read Δ as the primary quantity and p descriptively). The plain-JS arm is omitted: it ties TLA$^+$ in every generation (Δ=0, p=1).

Model	Unique (dep/con/plain/TLA$^+$)	JS(deployed) vs. TLA	JS(constrained) vs. TLA
Opus 4.8	2/1/1/1	Δ=.186, p=.0079*	Δ=.107, p=.0079*
Fable 5	2/2/1/1	Δ=.421, p=.0079*	Δ=.043, p=.44
Sonnet 4.6	1/1/1/1	Δ=.500, p=.0079*	Δ=0, p=1
Haiku 4.5	1/2/1/1	Δ=.714, p=.0079*	Δ=.593, p=.0079*

prescriptiveness remains a result in its own right — *it is a first-class experimental variable, large enough to masquerade as a language effect* — but on this task it is the model-dependent factor, and the contract is the uniform one.

The missing cell: the lean contract needs no semantics block. The lean-without-semantics arm reaches **100% for every generation of every model** — including Haiku, at 28.6% under the SAM contract with the same absent semantics. All 20 generations are textually distinct (5/5 unique per model, saved with the study artifacts) yet behaviorally identical and correct: given a bare `next()` target, every model derives the observable single-step semantics from the kernel source alone. This settles the conditionality the incomplete design left open: contract minimality buys transcription fidelity *unconditionally* on spec-level semantics in the prompt. Two honesty notes travel with this. First, with both lean cells and TLA$^+$ at 100%, effect *ordering* at the top is not identifiable — the within-lean prompt effect and any prompt×contract interaction have no headroom to appear (a ceiling effect); what is identifiable is that contract $\geq$ prompt for every model, strictly greater for Opus and Haiku, and that the lean contract suffices without semantics while the semantics block within SAM does not (three of four models below 100%). Second, the 100% ceiling itself is a property of this task's tiny observable relation; the `locksvc` replication (§8.4) shows the lean result generalizing to a richer state, but the factorial has not yet been re-run there.

The derivation arms split the languages — and the split is the oracle's paradigm, not the language. With the semantics block removed from both minimal contracts, lean JS reaches 100% for every model while TLA$^+$ falls to 78.6–100% (Δ=.214 for Opus and Sonnet, at the permutation floor; Table 5). The mechanism contains no wrong post-states: every TLA$^+$ shortfall window is *unscoreable*, because unguided models write the idiomatic guarded action (enabled only when the lock is free), and under this oracle a disabled action on a pinned contention pre-state has no successor. The lean contract's *totality* forbids that idiom structurally; TLA$^+$ permits it; and the semantics block's "held ⇒ unchanged" clause is precisely what converts the guard into a no-op branch. Scored conditionally — the paradigm-fair reading, per the oracle commitments stated in the design — the guard-idiom specifications are 100% correct on every window they can replay, and the languages tie here too. "Contract first, prompt second" therefore refines to: *the prompt matters exactly where the contract under-constrains* — and the unconditional split is a fact about the oracle's paradigm, reported as headline only because the benchmark's deployed

scoring is unconditional.

A real, one-directional residual survives the prompt control. With identical semantics in both prompts, TLA$^+$ reaches 100% conformance for *every model and every generation*, while constrained JS-SAM does so only for Sonnet. At the generation level the residual is large for Opus and Haiku (both at the permutation floor, p=.0079, read descriptively), vanishes for Sonnet, and for Fable does not replicate (Δ=.043, p=.44) — the pooled McNemar had starred Fable's deficit, and that specific claim was pseudo-replication, now withdrawn. The residual is largest for the weakest model (Haiku: 40.7% vs. 100%), and the direction is uniform: descriptively, no JS-SAM specification ever passed a window that its TLA$^+$ counterpart failed, in any arm. (When one arm is at 100% this is partly structural — see §10 — but the direction is uniform in the deployed arm as well, where the gap is significant for all four models.)

The plain-JS arm locates the residual: it is the contract, not the language. The fourth arm ties TLA$^+$ at 100% for every model and every generation — identical per-window outcomes in every paired generation-set (Δ=0; no test needed). Haiku is the cleanest single datum: the weakest model, which reaches only 40.7% through SAM's machinery, expresses the identical semantics perfectly as a bare transition function — in the same language. Three attributions fall out. *It is not the language*: same JavaScript, different shape, 100%. *It is not formality*: TLA$^+$ and plain-JS share the same shape — a minimal declarative single-step transition over exactly the observable state — and both sit at 100%; what the three-arm data suggested was "formal directness" is really *minimal declarative shape. It is the SAM contract's defect surface*: proposal/acceptor wiring, intent firing, and the mandatory auxiliary state (`threadStatus`, `callType`) — precisely where Experiment 1's `try_lock`-from-free and release bugs lived. Every moving part the contract obliges is transcription surface, and the tax falls hardest on the weakest model (Haiku pays 59 points; Sonnet pays nothing).

Verdict against the pre-registered outcomes. Not the strong "JS-SAM wins" (the direction is reversed), and not a tie at the backend level — but at the *language* level, a tie is exactly what the controlled comparison shows: JavaScript expresses the transition relation as reliably as TLA$^+$ once the specification shape is matched. The pre-registered outcome list did not anticipate this resolution; we note for transparency that the plain-JS arm was designed after the three-arm results were known (though scored blind, by the same mechanical replay, on the same corpus).

Design implication for JS-SAM. The Phase-3 fidelity gap is the cost of the SAM specification contract, not a language limit — and the two can be decoupled. The backend can offer a lean specification shape (`next(state, action, model)`, the JavaScript analogue of the TLA$^+$ relation), or score conformance against a plain transition core over only the observable variables, dropping the mandatory auxiliary state from the replay contract. SAM's ceremony is not gratuitous — the instance, intents, and value domains are exactly what feed the Phase-2/4 bounded explorer — but it should be paid where it buys something: explorable structure for model checking, not conformance replay. A prototype confirms the decoupling is practical: a ∼40-line generic breadth-first explorer over `{init, next}` runs the entire pipeline in the same locked-down sandbox, so the SAM machinery is not *required* by any phase. Two points keep this precise. First, the SAM checker itself was never the execution problem: it ran Phases 2 and 4 for every JS-SAM arm in this paper (the full depth-6 intent exploration each time) without failing on

any specification it was given; the tax is on the *authoring* side — the instance, intents, acceptors, and auxiliary state the model must hand-write to make a specification checker-consumable. ("Ran without failing" is itself a bounded claim: the metric audit in §5 shows the checker also ran to completion on a specification whose exploration never leaves its initial state, so a clean Phase-2 run certifies checkability, not adequacy.) Second, nothing is lost at this scale by the replacement: the SAM checker and the generic explorer are both bounded safety explorers (reachable states, invariants, deadlock), equivalent in checking power on these state spaces. The hybrid path keeps the library checker unchanged — the model writes only the pure `next`, the harness generates the scaffolding the checker consumes — relocating the checker from the specification contract to the tooling, which is where TLC has always lived on the TLA$^+$ side: no one asks the model to write TLC. A follow-up study closes the loop with model-generated specifications: each of the four models generated five lean specifications (N=5, 20 in total, each saved and evaluated as a single artifact across all phases), and **20 of 20 pass every phase** — loadable, Phase-2 clean, Phase-3 28/28, all three Phase-4 invariants holding. Two caveats travel with this result: `spin`'s observable state is tiny (three reachable states), so the lean explorer's adequacy must be re-validated on richer tasks; and the 20 generations collapse to roughly nine unique solutions — the models converge on the same correct `next` — which is expected for so small a system but limits the effective sample. Both caveats are resolved by a second task (§8.4).

Checkability, revisited. Zero unscoreable generations in the constrained TLA$^+$ arm confirms Experiment 3's attribution: the earlier "1/4 model-checkable" was entirely the unbounded-`CHOOSE` trap plus the configuration surface, both of which the pinned contract and direct replay remove. The checkability gap is real but shallow — a matter of guardrails, not capability.

A free variance check on Experiment 1. The JS(deployed) arm replicates Experiment 1's setting at N=5. The single-generation scores (89.3, 50.0, 50.0, 21.4) sit near the five-run means (81.4, 57.9, 50.0, 28.6), and the model ordering is preserved (with Fable's mean now slightly above Sonnet's). Experiment 1's qualitative conclusions survive replication; its exact figures move by up to ∼8 points, which is the error bar readers should apply to any single-generation number in this paper.

8.4 Generalizing the lean result: a second task

The lean-contract demonstration carried two caveats: a three-state observable space, and 20 generations collapsing to ∼9 unique solutions. To resolve both, we built a second full lean task around `locksvc`, SysMoBench's PGo-based distributed lock service — chosen over the ring-buffer candidate because it runs as an ordinary Go test with no kernel build, demonstrating in passing that the trace-capture methodology of §4 is not kernel-specific. The harness runs PGo's trace-recorder test five times and folds the raw MPCal events into observable windows over $\{holder, waiters\}$ — a lock holder plus the *set* of waiting clients — yielding a 60-window corpus (15 per action), self-consistent across runs. The set representation is itself a correction found by the base-rate audit: a first version modeled `waiters` as a FIFO in client-*send* order and required grants to go to the head, but the real server grants in *arrival* order, which the network reorders relative to send order (one capture logged sends $1, 3, 2$ while the server queued $\langle 3, 2, 1\rangle$ — the exact reordering the upstream specification's NoPriorityInversion comment warns about); the send-ordered fold silently converted ten real grants and releases into no-op windows. Arrival order is not a function of the client-side single steps, so at this projection the waiting clients form

a set and a grant may go to any waiter of a free lock; the corpus was re-folded (15 no-change windows remain, all legitimately the `CriticalSection` no-ops — identity base rate 25%) and the study regenerated from scratch. The fold also repairs a real trace artifact: PGo's empty vector clocks allow a `CriticalSection` event to be logged before its own `Grant`; the harness reorders each grant before its critical section, which is causally sound (the critical section cannot be entered without the grant). The replay and drivers were generalized to be task-agnostic: per-task action domains and invariants, and the projection rule in place of `spin`-specific keys.

The result replicates in full on the corrected corpus: **20 of 20 freshly generated lean specifications pass every phase** (5/5 per model, including the bounded progress checks), over a 20-state observable space with a genuine waiting set, and with 14/20 unique solution texts (4/4/4/2 per model) — materially different correct implementations, all conforming, 45/45 on the state-changing windows. Both caveats are thereby resolved: the clean sweep is not an artifact of a trivial state space, and the models do not converge on a single memorized solution. Across the two tasks, 40/40 model-generated lean specifications pass every phase.

9 Discussion

"Looks right" is not "is right," in any language. The four experiments repeatedly produced specifications that passed every internal-consistency check while being wrong about the real system — most vividly Fable's never-releasing lock, which preserved mutual exclusion *because* it was broken. Phase 3's value is not incremental rigor; it is the only phase whose ground truth is external to the model. This replicates SysMoBench's core claim [1, 2] in a new language substrate, which strengthens it: the phenomenon is about LLM modeling, not about TLA$^+$.

The familiar path is modeled; the unfamiliar path is not. For the three stronger models, every Experiment-1 failure lands on a less-common code path: the non-blocking `try_lock`-from-free acquisition (all three) and release bookkeeping (two of the three). Blocking acquisition — the version of a spinlock every textbook presents — was modeled correctly by all but the smallest model. This is the "textbook template" failure mode of [2] reproduced at the granularity of individual actions within a 100-line system.

Repair belongs in the evaluation loop. The repair experiment suggests that single-shot leaderboard scores understate some models (Fable) and overstate the ceiling of others (Haiku). Since practical formal-modeling deployments will be agentic loops with exactly this kind of counterexample feedback, benchmarks arguably should report both axes. The repair driver we contribute makes the second axis cheap to measure for any JS-SAM task.

The hypothesis, after Experiment 4: a split verdict, sharply located. The practical form of the hypothesis survives: host-language familiarity plus a self-contained executable artifact gets models to an evaluable specification far more often (4/4 vs. 1/4), and evaluability is the precondition for everything else. The strong form — that familiarity yields more *faithful* models — resolves to a tie once like is compared with like: JavaScript in the shape of the TLA$^+$ relation matches TLA$^+$ at 100% for every model. (The one cross-language split, in the derivation arms, favors lean JS and traces to the lean contract's totality against TLA$^+$'s guard idiom — not to the language; scored conditionally, the tie holds there too; §10.) What the three-arm data made look like a language effect was the specification contract: every layer of machinery between

the semantics and the artifact — proposals, acceptors, auxiliary state — is surface on which transcription slips, and the tax falls hardest on the weakest models. Neither training-data abundance nor formality is the operative variable on this task; *contract minimality* is. The largest single-model effect in the study is a prompt effect (Sonnet 50%→100%), but the completed factorial shows the prompt moves only two of four models within the SAM contract while the contract switch moves all four to ceiling with no semantics at all: contract, then prompt, then language is the order of what mattered here.

A benchmark everyone passes measures nothing — and that, too, is a finding. There is an irony in the lean-contract result that deserves stating plainly. This paper's first finding is that Phase 3 was the only phase that discriminated among models; its last is a specification shape under which every model — Haiku included — scores 100% on both tasks. The lean contract does not break the benchmark; it *saturates* these two tasks, and the saturation is itself informative: under a minimal contract — with or without the semantics in the prompt — specifying a system of this size is a solved problem for current frontier models. What saturation exposes is exactly what should be measured next: derivation at depth (systems whose observable relation cannot be guessed from action names or summarized in a prompt), scale, and the temporal properties no bounded safety explorer checks. A benchmark's job is to move its frontier when models catch up; the lean contract is that motion, not its defeat.

10 Limitations and threats to validity

We state these plainly; the study is a case study and should be read as one.

- **The controlled comparisons concern one system.** Experiments 1–4 all concern the spinlock, the *easiest* class of SysMoBench task; the lean-contract validation now spans two tasks (§8.4), but the factorial controlled comparison itself has not been repeated on the second. Distributed protocols with richer state (Raft variants) may reorder results. The spinlock's simplicity is also why Phases 1/2/4 saturate; on harder tasks they discriminate too [2].
- **Experiments 1–3 are single-generation.** Each cell in Tables 1–4 is one sample. Experiment 4's N=5 deployed-JS arm retroactively bounds the damage — ordering preserved, individual scores moving by up to ∼8 points — but the repair-loop and checkability experiments have not been replicated and should be read with that error bar.
- **Trace coverage is the quiet weakness of every conformance number here** (audited: `scripts/trace_coverage_audit.py`). The `spin` corpus is 28 windows from *two hand-authored deterministic scenarios*, covering 8 of the 18 observable (pre-state, action, data) combinations; contention is expressed only via `try_lock` (2 combos), and the corpus contains *zero* windows of a blocking `lock()` observed while the lock is held — the instrumentation emits the acquire event at acquisition success, where the pre-state is free, so the spin phase (the behavior the primitive is named for) is invisible in this two-variable projection by construction: a woken blocking waiter is indistinguishable from an uncontended acquire. The `locksvc` corpus is one fixed terminating workload (three one-shot clients; exactly 12 windows per run by protocol completion, hence the exactly-15-per-action count) sampled under five schedules — genuinely distinct (5/5 different action sequences, four different

grant orders, including the network reordering that exposed the fold bug), but with no re-requests, no deeper contention, and no failure paths. The vector-clock reorder's causal premise is *verified per event* at corpus build, not assumed: every `CriticalSection` step reads `GrantMsg` from its mailbox (message-passing happens-before), and the builder refuses traces where that check fails. Net: every 100% in this paper means conformance to a two-variable projection of an easy, deterministic slice of each system's behavior — faithful to real executions, far from exhausting them — and the conclusion carries this qualifier explicitly.

- **The TLA$^+$ comparison mixes causes.** The 1/4-vs-4/4 result bundles a genuine model-attributable pitfall (unbounded `CHOOSE`, 2/4) with a harness shortfall (config generation, 1/4). We report the attribution per model (Table 4); the headline number should not be read as purely a language effect. No repair round was run on the TLA$^+$ side — a repaired Opus might fix its `CHOOSE` readily, and symmetry demands that experiment before strong claims.

- **The derivation-level comparison is now run — and it splits the languages for the first time.** An earlier draft noted Experiment 4 measured transcription, not derivation, with no TLA$^+$-without-semantics arm to pair against the lean one. That arm now exists (`tld`: the constrained TLA$^+$ contract with the semantics block removed, mirroring the lean derivation arm). Result: in derivation mode the lean JS contract beats TLA$^+$ for Opus and Sonnet (Δ=.214, p=.0079 each; 100% vs. 78.6%), directionally for Haiku (87.1%, p=.17), and ties for Fable (100%). The mechanism is precise and is not wrong post-states: every TLA$^+$ shortfall window is *unscoreable* — without instruction, most models write the classic TLA$^+$ idiom in which `AcquireLock` is *guarded* on the lock being free, so a failed attempt is a disabled action rather than an observable no-op step, and the pinned contention windows have no successor. The lean contract's *totality* (`next()` must return a state) structurally forbids this; TLA$^+$'s idiom permits it, and the semantics block's "held $\Rightarrow$ unchanged" sentence turns out to be what fixes it (`tld` vs. `tla`: p=.0079 for both affected models). Under the pre-registered *conditional* scoring the guard-idiom specs are 100% correct on every window they can replay — the shortfall is replay-contract incompatibility, not incorrect modeling — so both numbers are reported, with the unconditional as headline. "Contract first, prompt second" thus refines to: *the prompt matters exactly where the contract under-constrains.*

- **The plain-JS arm resolves the substrate asymmetry, but was added post hoc and answers a narrower question.** The three-arm design left "executable machinery admits bugs a two-variable relation cannot" as an open alternative; the fourth arm was designed to test exactly that, after the three-arm results were known (its data were then collected by the same blind mechanical replay). Its conclusion has since been extended rather than bounded: a generic `{init, next}` explorer replaces the SAM scaffolding for Phases 2 and 4 (at equivalent bounded-safety checking power on these state spaces), and 20/20 model-generated lean specifications pass every phase on `spin` — but that demonstration inherits `spin`'s three-state observable space and a converged solution set ($\sim$9 unique among 20), so the finding needed a second task. The `locksvc` study (§8.4) supplies it — a 20-state space, 14/20 unique solution texts, 20/20 passing — generalizing the contract-tax removal across tasks of this scale, though not yet to large distributed protocols.

- b=0 **is partly structural, and the effective sample is small.** When the TLA$^+$ arm passes every window it fails none, so a JS-only pass is impossible by construction; any paired comparison reduces to whether the JS-SAM failure count is significant. Moreover the generations collapse to 1–2 unique behavioral fingerprints per (model, arm) — the pooled window-level McNemar we originally reported counted the same defect up to five times, and its χ^2 values are retracted in favor of the generation-level permutation test of Table 6. At the strictest unit (the unique solution, n=1–2 per cell) no significance test is meaningful; the per-solution effect sizes stand on their own.

- **The direct TLC replay is the controlled condition, not the deployed one.** The benchmark ships an agent-mediated TLA$^+$ Phase 3; pinning spec variables to the trace schema trades realism for comparability. Whether a "pinned-schema" Phase-3 mode should exist as a first-class controlled condition alongside the agent path is a methodological question we are putting to the maintainers.

- **Phase 4 for JS-SAM is safety-only and could not fail the headline bug in principle.** This is stronger than a template-count difference (seven TLA$^+$ templates vs. three JS-SAM). The three JS-SAM invariants are all safety properties over reachable states, and a never-releasing lock satisfies every one of them *because* it is broken: mutual exclusion holds trivially when the lock never changes hands, status consistency holds for a permanently locked holder, and the shipped `NoDeadlock` ("some thread is not `trying`") is satisfied by the stuck holder itself being `locked`. We verified this empirically: a release-is-a-no-op mutant of the known-good specification passes Phase 4 with the identical 3/3 verdict as the correct one. The properties that would discriminate — every waiter eventually acquires, every hold is eventually released — are liveness, and neither the SAM checker nor the lean explorer checks any temporal property (the JS-SAM template library states the exclusion explicitly). Fable's defect is doubly invisible: it manifests only under a pinned observable pre-state, so even a liveness-capable from-init checker would pass its specification — only trace replay catches it. As mitigation the lean explorer now supports *bounded progress* checks (EF-reachability: from every reachable state satisfying a premise, a goal state must be reachable within the bound — not liveness: no fairness, bounded horizon). These fail the never-releasing mutant (`ReleaseProgress`) and would fail the inert Haiku specification (`AcquireProgress`) that safety-only Phase 4 passed vacuously; all 40 saved model-generated lean specifications pass them on both tasks.

- **Model family.** All four models are from one vendor. The benchmark's configuration already includes other vendors' models; the comparison should be broadened before generalizing. A further reproducibility caution: several of the exact model versions used here may not be externally accessible; the saved specifications and raw per-window data, not the model identifiers, are the reproducible objects.

- **Missing arms, with status.** Review enumerated the acknowledged-but-load-bearing gaps; their current state: the *derivation arms* (no semantics block, both languages) are now run (see above — the one gap that, once filled, changed a conclusion); *multi-round repair* is now run (Haiku does not converge; §6); the *as-deployed agent-mediated TLA$^+$ Phase 3* remains unrun — it reintroduces the very confound the direct replay removed, and belongs in a separate agent-capability study; *other vendors* remain blocked on credentials (the registry entries exist in `config/models.yaml`; only Anthropic keys are available in

this environment); and the *factorial on* `locksvc` remains open (the lean arms exist; the SAM and TLA$^+$ arms need locksvc prompts and a locksvc TLC replay module). Each is inventoried with run instructions in `REPLICATION.md`.

- **Author positionality — and the case for independent replication.** The first author is the author of the SAM pattern. The evaluation pipeline is automated and the corpus is model-independent, which limits (but does not eliminate) the room for favorable design choices; and the study's sharpest negative finding — the contract-tax attribution — cuts *against* the SAM contract as deployed, which we take as evidence the pipeline is not tilted toward it. But the deeper issue, raised in review, is that the load-bearing materials on *both* sides of every comparison are experimenter-authored: the trace scenarios, the semantics block, the derivation prompts, and the repair prompt. No amount of automation neutralizes that. The repository therefore ships a replication guide (`REPLICATION.md`) that reruns every analysis from committed raw data without API access, regenerates every experiment from pinned prompts and models, and inventories each experimenter-authored material as an independent variable to audit or vary. We consider an independent replication by researchers unaffiliated with SAM — ideally the SysMoBench maintainers — a prerequisite for treating the cross-language conclusions as more than a well-documented single-author case study.

11 Related work

Benchmarking LLM formal modeling. SysMoBench [1, 2] is the direct foundation of this work: it contributes the four-phase methodology, the eleven-system task suite, and the finding that conformance — not syntax — is where LLM specifications fail. Our contribution is orthogonal: holding the benchmark fixed and varying the specification *language*, including the first non-formal one. The Specula agent [13] shows that agentic workflows can saturate current SysMoBench tasks in TLA$^+$; our repair experiment probes the smallest unit of such a workflow (one counterexample round) across models.

The wider LLM×formal-methods field. A relevance-ranked sweep of the 2022–2026 literature (40 works; the map ships with the artifacts) places this study in a thin slice of a fast-moving field. The center of gravity is autoformalization and theorem proving — Lean/Isabelle proof generation and repair [14, 15, 16] — followed by loop-invariant and precondition generation for deductive verification [18]. Specification-focused work concentrates on natural-language-to-temporal-logic translation [17] and declarative-specification repair [20]; benchmarks are proliferating (e.g., OS-VBench for OS-verification specification generation [19]) but grade against reference specifications, proof success, or human judgment rather than execution-trace ground truth. Against that corpus, this study's two distinctive commitments — conformance to traces captured from the running system as the discriminating oracle, and a controlled contract×prompt×language factorial — are both minority positions. The closest contemporaneous works, an NL-to-TLA$^+$ correctness evaluation and counterexample-driven TLA$^+$ repair [21, 22], each parallel a single axis of the design here (one-shot generation; repair); the repair-as-separate-axis finding of Experiment 2 matches an emerging pattern across proof repair [16] and specification repair [20].

Trace conformance for specifications. Validating specifications against implementation traces has industrial precedent: MongoDB's eXtreme modeling checked TLA$^+$ models against

driver traces [8], and trace validation is the standard answer to "does the spec match the code" [1]. SysMoBench's transition-window formulation, which we adopt unchanged, makes conformance per-action and per-window rather than whole-trace.

Formal methods practice. TLA$^+$ [3] with TLC [4], Alloy [5], and CSP-based PAT [6] are the benchmark's formal backends; industrial deployments (e.g., [7]) motivate the whole enterprise. SAM [9, 10] occupies an unusual position: an engineering pattern with TLA$^+$-derived semantics, here repurposed as a specification language precisely because it sits at the intersection of "formally disciplined" and "abundant in training data."

12 Conclusion and future work

We extended SysMoBench with JS-SAM, its first non-formal specification backend, built the kernel trace-capture pipeline needed to give it a real conformance phase, and ran four experiments on the Asterinas spinlock task, extending the lean-contract validation to a second task (a distributed lock service). The results make four points with unusual precision for a single case study: transition validation against real traces is the only phase that discriminated among four frontier models (0–86.4% on state-changing windows, with all other phases unanimous and the weakest model sitting exactly at the corpus's identity base rate); one round of trace-feedback repair reveals a self-correction axis that one-shot scores conceal — capable models repair fully and verifiably (a held-out audit found root-cause fixes, not memorized windows), the weakest not at all; the same models that reach an evaluable specification in JavaScript 4/4 times manage it in TLA$^+$ 1/4 times — for two identified causes, one of them a harness fault, both later removed entirely by a one-line prompt guardrail: a default-prompt robustness gap, not a language-capability one; and under a pre-registered, paired comparison completed to a full 3×2 factorial in (contract, prompt), the faithfulness question resolves in an unexpected place — the conformance gap is attributable to the SAM contract's machinery, not to JavaScript, not to formality, and not primarily to prompt prescriptiveness: plain JavaScript in the shape of the TLA$^+$ relation ties TLA$^+$ at 100% for every model, with or without spec-level semantics in the prompt.

The hypothesis that motivated the backend is neither confirmed nor dead; it is answered more usefully than either. Stated plainly, on the tasks tested the completed study ranks what governed LLM specification quality: *contract shape first, prompt second, language nowhere* — a ranking conditional on the spinlock factorial until replicated on a richer task. Familiarity buys *checkability*; contract minimality buys *transcription fidelity* — unconditionally: the lean-without-semantics cell reaches 100% for every model, including the weakest at 28.6% under the SAM contract, so every model *derives* the correct observable semantics from the kernel source when the target is a bare transition function. The prompt is the model-dependent factor: four lines of semantics moved Sonnet 50%→100% and Fable 57.9%→95.7% within the SAM contract, but moved Opus and Haiku insignificantly — and at the lean contract it had nothing left to move (a ceiling at which effect ordering among the perfect cells is, we note, not identifiable). Every layer of ceremony between the semantics and the artifact cost conformance; the language itself, once shape was matched, contributed nothing measurable (the derivation-mode split between lean JS and TLA$^+$ traces to contract totality, not language: scored conditionally, the guard-idiom TLA$^+$ specifications are perfect on every scoreable window). On the practical question — can JavaScript replace TLA$^+$? — the evidence splits cleanly. For LLM-generated conformance modeling at this scale: yes — interchangeable on fidelity *on what we measured*, which is conformance to a

two-variable observable projection of deterministic, easy slices of each system's behavior (§10); neither language has been tested on uncovered combinations, projection-invisible behavior such as the blocking spin, richer workloads, or adversarial interleavings. Within that measured slice the tie is exact, and JavaScript carries fewer operational failure modes (no `CHOOSE` trap, no configuration artifact). For deep verification: no — TLC's temporal properties, liveness checking, and large-state model checking have no equivalent in either bounded safety explorer used here (the SAM library's checker or its 40-line generic replacement), and remain TLA$^+$'s distinctive value. The search this motivates is therefore not for a better specification *language* but for a better specification *contract*: `{init, next}` over observable keys, in whatever language the model writes well, with richer machinery generated by the harness rather than transcribed by the model. Two questions remain genuinely open: whether these conclusions survive *derivation at depth* — the derivation arms already run show every model deriving this lock's semantics unaided in the lean shape, and most models falling into TLA$^+$'s partial-action idiom without guidance, but a spinlock's observable relation is guessable in a way a consensus protocol's is not — and whether they survive scale.

The experiments that follow are concrete: (1) promoting the lean specification shape from demonstration to backend: with model-generated lean specifications passing all phases 40/40 across both tasks, the remaining question is purely integration — a first-class lean backend, a hybrid where the model writes only the pure core and the harness generates the SAM scaffolding, or the documented status quo; (2) derivation at depth: with the `spin` derivation arms now run, the decisive open test moves to a task whose semantics cannot be guessed from the action names — the `locksvc` factorial (its SAM and TLA$^+$ arms are scoped in the replication guide) and, beyond it, a consensus-class protocol; (3) a prompt-prescriptiveness dose–response study, treating the semantics block as the manipulated variable rather than a nuisance; (4) an as-deployed agent-path arm beside the controlled replay, for ecological validity; (5) a TLA$^+$ repair round for symmetry with Experiment 2; and (6) replication beyond one model family. We are contributing the backend, harnesses, drivers, corpora, and the controlled-comparison methodology upstream so these can be run — and challenged — by the community. One such test is already underway as this is written: a real-world study on a production-grade distributed state machine — point of sale, payment terminal, and payment backend across a payment transaction's lifecycle — is running the same JS-SAM-vs-TLA$^+$ comparison, and will be reported separately.

Acknowledgments

We thank the SysMoBench / Specula maintainers for the benchmark, the integration guidance that shaped the JS-SAM backend's scope, and the reference instrumentation for the Asterinas spinlock.

Reproducibility

All artifacts are public: the JS-SAM backend (`tla_eval/languages/js_sam.py`, `tools/js-sam/`), the trace harness (`scripts/harness/spin/`), the captured corpus (`data/sys_traces/spin/`), the repair driver (`scripts/repair_phase3.py`), the Experiment-4 apparatus — the constrained and plain-JS prompts, the direct TLC replay and its functional-check audit (`scripts/tla_direct_tv.py`, the 20 audited specifications in `output/tla_specs/`, `output/tla_functional_audit.json`, the permissive control in `tools/tla/`), the paired factorial driver with raw per-window data

(`scripts/tla_phase3_study.py`, `output/tla_phase3_study.json`), the generation-level analysis of record and the repair-generalization audit (`scripts/tla_phase3_analysis.py`, `scripts/repair_generali`
the trace-coverage audit (`scripts/trace_coverage_audit.py`), the pre-registered design and results (`docs/js_sam_tla_phase3_*.md`), the lean-contract prototype and full-pipeline studies (`tools/plain-js/`, `scripts/plain_js_tv.py`, `scripts/lean_full_pipeline_study.py`, the 40 saved generations in `output/lean_specs*/`, `docs/js_sam_lean_contract.md`), the `locksvc` harness and corpus (`scripts/harness/locksvc/`, `data/sys_traces/locksvc/`), and the methodology note (`paper/methodology.md`), and the literature map (`paper/literature_map.md`) — and the experiment write-ups (`docs/js_sam_*.md`) in the SysMoBench fork at `https://github.com/jdubray/SysMoBench-1`, staged as stacked pull requests against `https://github.com/specula-org/SysMoBench`. Requirements: Docker, Node ≥ 20, Python 3, and (for the TLA$^+$ arm) Java with `tla2tools`. A single script reproduces the kernel capture: `scripts/harness/spin/run.sh`. A replication guide (`REPLICATION.md`, repository root) re-derives every analysis from committed raw data without API access, regenerates every experiment from pinned prompts and models, and inventories the experimenter-authored materials as independent variables to audit or vary.

References

[1] Q. Cheng, R. Tang, E. Ma, F. Hackett, P. He, Y. Su, I. Beschastnikh, Y. Huang, X. Ma, and T. Xu. SysMoBench: Evaluating AI on formally modeling real-world systems. arXiv:2509.23130, 2025. Leaderboard: `https://sysmobench.com`.

[2] Q. Cheng, R. Tang, E. Ma, F. Hackett, P. He, Y. Su, I. Beschastnikh, Y. Huang, X. Ma, and T. Xu. Can LLMs model real-world systems in TLA+? ACM SIGOPS Blog, May 2026. `https://www.sigops.org/2026/can-llms-model-real-world-systems-in-tla/`.

[3] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[4] Y. Yu, P. Manolios, and L. Lamport. Model checking TLA+ specifications. In *CHARME*, 1999.

[5] D. Jackson. Alloy: A lightweight object modelling notation. *ACM TOSEM*, 11(2), 2002.

[6] J. Sun, Y. Liu, J. S. Dong, and J. Pang. PAT: Towards flexible verification under fairness. In *CAV*, 2009.

[7] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4), 2015.

[8] A. J. Davis, M. Hirschhorn, and J. Schvimer. eXtreme modelling in practice. *PVLDB*, 13(9), 2020.

[9] J.-J. Dubray. The SAM pattern. `https://sam.js.org`.

[10] J.-J. Dubray. Why I no longer use MVC frameworks. InfoQ, February 2016. `https://www.infoq.com/articles/no-more-mvc-frameworks/`.

[11] J.-J. Dubray. `sam-pattern`: a library implementing the SAM pattern. `https://www.npmjs.com/package/@cognitive-fab/sam-pattern`.

[12] Asterinas project. Asterinas: a Linux ABI-compatible, Rust-based framekernel OS. `https://github.com/asterinas/asterinas`.

[13] Specula project. Specula: an agent specialized in TLA+ formal modeling. `https://github.com/specula-org/Specula`.

[14] Y. Wu, A. Q. Jiang, W. Li, M. Rabe, C. Staats, M. Jamnik, and C. Szegedy. Autoformalization with large language models. In *NeurIPS*, 2022.

[15] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS*, 2023.

[16] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. In *ESEC/FSE*, 2023.

[17] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *CAV*, 2023.

[18] S. Chakraborty et al. Ranking LLM-generated loop invariants for program verification. In *Findings of EMNLP*, 2023.

[19] OSVBench: Benchmarking LLMs on specification generation tasks for operating system verification. arXiv:2504.20964, 2025.

[20] An empirical evaluation of pre-trained large language models for repairing declarative formal specifications. *Empirical Software Engineering*, 2025. doi:10.1007/s10664-025-10687-1.

[21] Can LLMs write correct TLA+ specifications? Evaluating natural-language-to-TLA+. 2026. Contemporaneous work, located via literature search.

[22] TraceFix: Repairing agent coordination protocols with TLA+ counterexamples. 2026. Contemporaneous work, located via literature search.